

开源鸿蒙（OpenHarmony）编码规范

本文档基于 OpenHarmony 官方 C 语言编程规范，并结合本仓库（ws63_ohos）源码实践整理。

官方规范：<https://gitee.com/openharmony/docs/blob/master/zh-cn/contribute/OpenHarmony-c-coding-style-guide.md>

文档更新于 2026-03-12，为本人在开发中总结的经验，如有问题请及时联系王老师（wangpeizheng@sztu.edu.cn）。

一、总体原则

代码需在保证功能正确的前提下，满足**可读、可维护、安全、可靠、可测试、高效、可移植**的要求。

- 规则：**必须遵守
- 建议：**应尽量遵守
- 例外：**有充分理由时可适当违背，但应尽量保持风格一致

二、命名规范

2.1 基本风格

类型	命名风格	示例
函数、结构体、枚举、联合体	大驼峰 (UpperCamelCase)	GpioLedTask, GetUserInfo
变量、参数、结构体成员	小驼峰 (lowerCamelCase)	count, ledGpio
宏、常量、枚举值	全大写 + 下划线	LED_GPIO, TICK_PER_SEC

2.2 特殊规则

- 全局变量：**加 `g_` 前缀，如 `g_activeConnectCount`
- 文件命名：**小写字母、数字、下划线，如 `gpio_led.c`
- 函数命名：**动宾结构或形容词，如 `AddTableEntry`、`IsRunning`

2.3 本仓库约定

- 新建 C 源码头部版权块后添加：`* 适用平台：九联星闪开发板，海思 WS63E 芯片。`
- 入口函数命名：`xxxEntry`（如 `GpioLedEntry`、`HelloWorldEntry`）

三、排版格式

3.1 缩进与行宽

- **缩进**: 4 个空格, 禁止 Tab
- **行宽**: 不超过 120 字符

3.2 大括号 (K&R 风格)

```
int Foo(int a)
{
    if (condition) {           // 函数左大括号独占一行
        DoSomething();         // 其他左大括号跟语句放行末
    } else {
        DoOther();
    }
}
```

3.3 条件与循环

- 条件、循环**必须**使用大括号, 即使只有一条语句
- `if/else/else if` 不得写在同一行

```
if (condition) {
    return CreateNewObject();
}

for (int i = 0; i < range; i++) {
    DoSomething();
}
```

3.4 空格

- `if`、`for`、`while` 等关键字后加空格
- 二元操作符两侧加空格
- 逗号、分号后加空格, 前不加
- 小括号内侧不加空格

```
if (x && !y) {
    v = w * x + y / z;
}
```

四、版权与文件头

4.1 文件头注释（必须包含版权）

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
```

4.2 BUILD.gn 文件头

```
# Copyright (c) 2026 SZTU Open Source Organization.
# Licensed under the Apache License, Version 2.0 (the "License");
# ...（同上 Apache 2.0 全文）
```

五、应用入口与任务

5.1 入口注册

- 使用 `APP_FEATURE_INIT(EntryFunc)` 或 `SYS_RUN(EntryFunc)` 注册应用入口
- 入口函数为 `static void xxxEntry(void)` 或 `static void xxxEntry(void *arg)`

```
static void GpioLedEntry(void)
{
    // 创建任务、初始化等
}

APP_FEATURE_INIT(GpioLedEntry);
```

5.2 任务创建

```
osThreadAttr_t attr = {
    "TaskName", 0, NULL, 0, NULL, STACK_SIZE, PRIO, 0, 0
};

if (osThreadNew(TaskFunc, NULL, &attr) == NULL) {
    printf("[Module] Failed to create task!\r\n");
}
```

5.3 未使用参数

使用 `(void)arg;` 显式忽略未使用的参数，避免编译告警。

六、头文件与依赖

6.1 包含顺序

- 1. 标准库: `#include <stdio.h>`
- 2. 空行
- 3. 项目头文件: `#include "ohos_init.h"`、`#include "iot_gpio.h"` 等

6.2 常用头文件

头文件	用途
<code>ohos_init.h</code>	<code>SYS_RUN</code> 、 <code>APP_FEATURE_INIT</code> 入口宏
<code>cmsis_os2.h</code>	线程、延时等 RTOS API
<code>iot_gpio.h</code>	GPIO 操作
<code>iot_watchdog.h</code>	看门狗喂狗

七、注释

7.1 基本原则

- **注释与代码同等重要**，修改代码时必须同步更新注释
- **按需注释**：解释意图、约束、陷阱，而非重复代码本身
- **简洁无歧义**：内容明确、信息完整、避免冗余
- **从读者角度**：写注释时考虑读者真正需要的信息

7.2 必须注释的场合

场合	要求
模块对外接口头文件	对每个函数声明进行注释
全局变量	说明用途、取值范围、使用约束
核心算法	说明思路、关键步骤或公式
超过 50 行的函数	在函数上方说明功能、参数、返回值
非显而易见的逻辑	说明为何这样写、有何限制

7.3 函数头注释

- 放在函数声明或定义**上方**
- 函数名能自说明时可省略；需要时补充：功能、返回值、内存约定、可重入性等
- **禁止**空有格式、无实质内容的注释

```
/* 返回实际写入的字节数，-1 表示失败；buf 由调用者释放 */  
int writeString(char *buf, int len);
```

```
// 错误示例：信息冗余、无实质内容  
/*  
 * 函数名：writeString  
 * 功能：写入字符串  
 * 参数：buf 缓冲区，len 长度  
 * 返回值：无  
 */
```

7.4 代码注释位置与格式

- 注释放于对应代码**上方**或**右侧**
- 上方注释与代码保持相同缩进
- 右侧注释与代码之间至少 1 空格，建议 2~4 空格
- 注释符与注释内容之间留 1 空格

```
/* 上方单行注释 */  
DoSomething();  
  
int foo = 100; // 右侧注释  
  
/*  
 * 上方多行注释  
 * 说明复杂逻辑或意图  
 */  
ComplexLogic();
```

7.5 宏与常量注释

- 宏、常量应在**上方**或**行末**注释说明用途和单位
- 魔数（如 500、8）应通过常量替代并加注释

```
/* GPIO10：九联星闪开发板通过排针与 LED 相连，低电平点亮 */  
#define LED_GPIO 10  
  
#define TICK_PER_SEC 100 /* WS63 系统 tick 频率 */  
#define MS_TO_TICKS(ms) (((ms) * TICK_PER_SEC) / 1000)  
  
/* 每 3 次闪烁喂狗一次（约 3 秒），防止看门狗超时 */  
if ((count % 3) == 0) {  
    IoTWatchDogKick();  
}
```

7.6 注释风格与语言

- 可使用 `/* */` 或 `///`，同一类型注释保持统一风格
- 本仓库**允许使用中文注释**，便于团队协作；对外接口建议中英双语或英文

7.7 禁止的注释

- 不要重复代码已表达的信息
- 不要写无内容的占位注释（如 `// TODO` 无说明）
- 不要保留已失效的注释（删除或修改代码后应同步删除/更新）

八、BUILD.gn 规范

8.1 应用模块结构

```
static_library("target_name") {
    sources = [ "source.c" ]

    include_dirs = [
        "//commonlibrary/utils_lite/include",
        "//kernel/liteos_m/components/cmsis/2.0",
        "//base/iothardware/peripheral/interfaces/inner_api",
    ]
}
```

8.2 目标名

- 与目录名对应，如 `gpio_led` 目录下目标为 `gpio_led_demo`
- 需在 `app/BUILD.gn`、`config.py`、`ohos.cmake` 中同步配置

九、WS63 平台注意事项

9.1 看门狗

长循环任务需定期调用 `IoWatchDogKick()`，否则约 8 秒后触发 NMI 复位。

```
#include "iot_watchdog.h"

while (1) {
    // ...
    if ((count % 3) == 0) {
        IoWatchDogKick();
    }
}
```

9.2 延时

WS63 系统 tick 为 100Hz (1 tick = 10ms) , `osDelay(n)` 参数为 tick 数, 非毫秒。

```
#define TICK_PER_SEC 100
#define MS_TO_TICKS(ms) (((ms) * TICK_PER_SEC) / 1000)

osDelay(MS_TO_TICKS(500)); // 500ms
```

9.3 串口输出

- 使用 `\r\n` 换行
- 避免循环内频繁 `printf`, 以免串口阻塞影响喂狗

十、参考

- [OpenHarmony C 语言编程规范](#)
- [04-创建新项目的方法.md](#)
- [06-GPIO的使用.md](#)